

Summary

The algorithms are a design technique used primarily for optimization problems, making a sequence of locally optimal decisions to reach a solution.

Critical Takeaways:

- **Methodology:** Greedy algorithms make step-by-step decisions, selecting the "best" available choice at each stage based on local information.
- **Core Logic:** Given an input set of n elements, the goal is to find a subset that satisfies constraints while optimizing an objective function.
- **Irreversibility:** Once an element is included in the solution, it cannot be removed or altered.
- **Proof of Correctness:** Local optimality does not guarantee global optimality, so correctness proofs are necessary.
- **Applications:** Common applications include Fractional Knapsack Problem (optimized via profit-per-unit-weight ratios) and Job Scheduling with Deadlines (optimized by scheduling high-profit jobs as late as possible within deadlines).

1. Conceptual Framework of Greedy Algorithms

1.1 Fundamental Definition

A greedy algorithm solves a problem by making a sequence of decisions, choosing the option that appears best at the current moment.

1.2 Local vs. Global Paradox

- **Local Solution:** Based on immediately available information.
- **Global Solution:** The ultimate objective (e.g., maximum total profit).
- **Analytic Challenge:** A local solution may not produce a global optimum, requiring proof of correctness.

1.3 Operational Characteristics

- **Input:** Set of elements/objects.
- **Step-by-step selection** of elements.
- **Decisions** are final once made.

2. Problem Categorization and Terminology

2.1 Problem Types

1. **Decision Problems:** Require a Yes/No answer (e.g., "Is n prime?").

2. Optimization Problems: Require maximizing or minimizing an objective; most greedy algorithms target these.

2.2 Key Terminology

Term	Definition
Solution Space	Set of all possible solutions to a problem.
Feasible Solution	A solution satisfying all given constraints.
Optimal Solution	A feasible solution that optimizes the objective function (max profit or min cost).

3. General Greedy Strategy (Basic Idea)

1. Initialize solution as empty.
2. Iterate through the input set of n elements.
3. Select an element from the input.
4. Feasibility Check: Verify if adding the element maintains feasibility (satisfies constraints).
 - Example: In Kruskal's Algorithm, add an edge only if it does not form a cycle.
5. Union: If feasible, include the element in the solution.
6. Return the final solution set.

4. The Fractional Knapsack Problem

4.1 Problem Definition

Maximize total profit without exceeding knapsack capacity (M) given n objects with weight (w_i) and profit (p_i).

4.2 Mathematical Formulation

- Objective Function: Maximize $(\sum_{i=1}^n (p_i \cdot x_i))$
- Constraint: $(\sum_{i=1}^n (w_i \cdot x_i) \leq M)$
- Decision Variable: $(x_i \in [0, 1])$, fraction of object i taken

4.3 Strategy Comparison

Strategy	Description	Outcome
Greedy by Profit	Select items with highest profit first	Suboptimal

Greedy by Weight	Select lightest items first	Suboptimal
Greedy by Profit/Weight	Select items with highest profit per unit weight	Optimal strategy

Optimal Approach: Sort items by decreasing (p/w) ratio and select accordingly.

5. Job Scheduling with Deadlines

5.1 Problem Constraints

- n jobs, each with burst time = 1.
- Single CPU, non-preemptive execution.
- Each job has profit (p_i) and deadline (d_i).
- Profit is earned only if job completes before its deadline.

5.2 "As Late as Possible" Strategy

- Insight: Scheduling a late-deadline job too early may block jobs with stricter deadlines.
- Greedy Algorithm:
 1. Sort jobs in decreasing order of profit.
 2. For each job, schedule in the latest available slot $\leq$ deadline.
 3. This ensures earlier slots remain free for jobs with tighter deadlines, maximizing total profit.